SpendGuard

1. Introduction

Purpose

This document explains the requirements of SpendGuard, a website that helps a person or a small team to understand their spending. The user uploads bank transactions as a CSV file, and the website finds subscriptions and recurring payments and reminds the user before they are renewed.

Scope

SpendGuard imports transactions from a CSV file, cleans the merchant names, puts each transaction in a category, finds recurring payments, shows monthly spending trends and a renewal calendar, and gives alerts for upcoming renewals.

Live bank connection, PDF statements and multiple currencies are not included in this project.

Definitions

- Transaction: one row of the CSV file with date, description and amount.
- Merchant: the shop or company where the payment was made, for example Netflix.
- Recurring payment: a payment that repeats again and again after a fixed time, like a monthly subscription.
- Renewal: the next date on which a recurring payment is expected.
- CSV: a simple file format used to export data from a bank.
- SQLite: a simple database that stores all data in a single file.

2. Overall Description

Product Perspective

SpendGuard is a web application. The frontend is made with HTML, CSS and JavaScript, the backend is made with Node.js and Express, and the data is stored in an SQLite database. A scheduled job runs every day on the server to create renewal alerts.

Product Functions

- Register and login
- Upload a CSV file of transactions
- Clean merchant names and give categories
- Find recurring payments and subscriptions
- Show spending trends and category wise spending
- Show renewal calendar and give alerts

User Classes

- Individual user: tracks his own spending and subscriptions.

- Team user (optional): a small team tracking shared tool subscriptions with one shared account.

Operating Environment

The website works in common browsers like Chrome, Edge and Firefox. The server needs Node.js. The SQLite database is a single file, so no separate database server is needed.

Constraints

- The project uses HTML, CSS, JavaScript, Node.js, Express and SQLite.
- Amounts are in rupees.
- Categories and recurring payments are found using simple rules, so no paid AI service is needed.
- The project has to be completed in a short time.

Assumptions

- The user can download a CSV file from his bank.
- The CSV file has at least a date, a description and an amount.
- At least three months of data is available to find monthly subscriptions.

3. Specific Requirements

The requirements are divided into six modules. Points marked (optional) will be done only if time is left.

Authentication

- A user can register with name, email and password.
- A user can login with email and password.
- Passwords are stored in a safe (hashed) form.
- Every user can see only his own data.
- A user can delete his account and data (optional).

CSV Import

- A user can upload a CSV file.
- The website finds the date, description and amount columns by itself, and the user can correct them if wrong.
- Common date formats are supported.
- Wrong rows are skipped and shown in an error list.
- If the same file is uploaded again, duplicate transactions are skipped.
- Only expenses (debits) are used for analysis.
- A preview of the data is shown before importing (optional).
- An import history is shown (optional).

Merchant Cleaning and Categories

- The merchant text is cleaned by removing extra words like UPI or POS, numbers and special characters.
- Different names of the same merchant are matched to one proper name, for example "netflix com" and "netflix india" become Netflix.
- Each transaction is given a category like Food, Shopping, Entertainment, Bills, Transport, Health, Subscriptions or Other.
- If no category matches, it is set to Other and marked for review.
- A user can change the merchant name or category of a transaction (optional).

Recurring Payment Detection

- Transactions of the same merchant are grouped together.
- The website checks the gap between payments to find weekly, monthly, quarterly or yearly payments.
- A few days of difference is allowed, so payments with irregular dates are also found.
- A small change in amount is allowed. A big change is shown as a price change.
- The next expected date and monthly cost are calculated for each recurring payment.
- The user can confirm, remove or add a recurring payment (optional).
- Detection runs automatically after every import.

Dashboard and Trends

- The dashboard shows total spending of this month and a chart of the last few months.
- It shows spending by category.
- It shows the number of subscriptions and their total monthly cost.
- A transactions page shows all transactions with search and filters.
- A category spike is shown when spending of a category suddenly increases (optional).
- Unusual payments are shown, for example when a subscription price increases (optional).

Renewal Calendar and Alerts

- A calendar shows the expected renewal date of each recurring payment.
- A list shows the renewals coming in the next 30 days.
- A scheduled job runs every day and creates an alert a few days before the renewal.
- The same alert is not created twice.
- The user can set how many days before the alert should come (optional).
- Email alerts (optional).

4. External Interface Requirements

User Interface

The website has these pages: Register, Login, Import, Dashboard, Transactions, Subscriptions, Renewal Calendar and Alerts. It should work on both mobile and computer screens.

CSV Format

The CSV file should have columns like Date, Description and Amount, for example: 05/08/2026, NETFLIX.COM 8007 CA, 649.00. Column names can be different for different banks, so the user can choose the columns while importing.

Software Interface

- SQLite database
- node-cron for the daily alert job

5. Non-Functional Requirements

- Performance: the dashboard should load in about 3 seconds and a normal CSV file should be imported quickly.
- Security: passwords are hashed, login is checked with a token, every user can access only his own data and secret keys are kept in the .env file.
- Reliability: one wrong row should not stop the whole import, and the daily job should not create duplicate alerts.
- Usability: the website should be simple, easy to use on mobile and show clear error messages.

6. Database Design

- User: name, email, password.
- Transaction: user, date, amount, original description, merchant, category.
- RecurringPayment: user, merchant, interval, amount, next expected date, status.
- Alert: user, recurring payment, renewal date, message, read status.

7. Future Scope

- Using AI to find categories of unknown merchants
- Connecting directly with bank accounts
- Email and push notifications
- Budget limits for each category
- Reading PDF bank statements